17

Floating Point and Stability

Historical Note

For a while it was believed that Gaussian elimination could not work. Hotelling published an error analysis in 1943 whose bound grew like 4^n, which would make a system of ten equations untrustworthy and a system of fifty absurd; von Neumann and Goldstine's careful study of 1947 was also discouraging. Yet people were solving such systems and getting right answers. Turing, in a 1948 paper on rounding errors, introduced the quantity he called the condition number and located part of the difficulty in the problem rather than the method. The decisive reframing came from James Wilkinson around 1960: instead of asking how far the computed answer is from the true one, ask what problem the computed answer solves *exactly*. Almost always it is a problem very close to the one posed, and the two questions together explain both why elimination works and when it does not. Wilkinson received the Turing Award in 1970. William Kahan, who did more than anyone to make floating point arithmetic behave predictably enough for such analyses to be possible, received it in 1989, four years after the IEEE 754 standard he shaped was published.

Every chapter so far has contained a warning about computation: do not invert, do not form A^tA, do not build the characteristic polynomial, pivot before you eliminate. Each was stated where it arose and justified informally. This chapter supplies the framework those warnings belong to. It has two halves, and the point is that they are different: some difficulties belong to the problem and no algorithm can avoid them, while others belong to the algorithm and a better one removes them.

17.1 Floating point numbers

Definition 17.1 (The floating point model). *A binary floating point system of precision p represents numbers as*

$$\pm (1.b_1 b_2 \cdots b_{p-1})_2 \times 2^e,$$

with each $b_i \in \{0, 1\}$ and e in some finite range. The unit roundoff *is $\varepsilon = 2^{-p}$. For IEEE double precision $p = 53$, so*

$$\varepsilon = 2^{-53} \approx 1.11 \times 10^{-16},$$

and e runs over roughly ± 1024, giving magnitudes from about 10^{-308} to 10^{308}.

Some authors write $\varepsilon_{\text{mach}}$ for 2^{-52}, twice our value; nothing below depends on the factor of two.

Key Idea

The fundamental axiom of floating point arithmetic. Every real number x in range has a nearest float $\text{fl}(x)$ with

$$\frac{|\text{fl}(x) - x|}{|x|} \le \varepsilon,$$

and every arithmetic operation, performed on floats, returns the correctly rounded exact result:

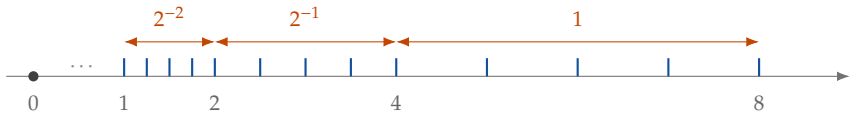
$$\text{fl}(x \odot y) = (x \odot y)(1 + \delta), \qquad |\delta| \le \varepsilon,$$

for $\odot$ any of $+, -, \times, \div$. Every error bound in this chapter is derived from that one sentence, and the reason it can be assumed is IEEE 754.

Figure 17.1 draws a toy system to scale, and the one thing to take from it is that the numbers are not evenly spaced. Each octave holds the same count, so the absolute gap doubles as the numbers grow while the relative gap stays put, which is why every bound in this chapter is a relative one.

Caution

Three things ε is not. It is not the smallest positive number: that is around 10^{-308}, and smaller still with subnormals. It is not an absolute error bound: the gap between neighbouring floats near 10^6 is about 10^{-10}, and near 10^{-6} about 10^{-22}.



four numbers per octave, so the gap doubles at every power of two

Figure 17.1. The floating point numbers of a toy system with a three-bit significand, drawn to scale. They are not evenly spaced: each octave $[2^e, 2^{e+1})$ holds the same count, so the absolute gap doubles as the numbers grow while the *relative* gap stays put at about ε. Below 1 the same pattern continues downwards for ever, the numbers crowding towards the origin without ever arriving; the dots stand for those octaves. This is why the fundamental axiom is a statement about relative error, and why ε measures spacing rather than smallness.

And it is not a bound you may add up naively over a long computation, because errors can cancel as well as accumulate.
Floating point addition is also not associative. With $u = 2^{-53}$, the sum $1 + u$ rounds to exactly 1, so $(1 + u) + u = 1$; but $u + u = 2^{-52}$ is exact and $1 + 2^{-52}$ is representable, so $1 + (u + u) \neq 1$. Reordering a sum can change the answer, which is why parallel reductions are not bit-reproducible.

17.2 Cancellation

The single most common way to lose accuracy is to subtract two nearly equal numbers.

Example 17.2 (Catastrophic cancellation). *Evaluate* $f(x) = \dfrac{1 - \cos x}{x^2}$ *at* $x = 10^{-8}$.
The true value is close to $\frac{1}{2}$, *since* $1 - \cos x = \frac{x^2}{2} - \frac{x^4}{24} + \cdots$.
In double precision, $\cos(10^{-8})$ *differs from* 1 *by about* 5×10^{-17}, *which is less than half the gap between* 1 *and its neighbour. So* $\mathrm{fl}(\cos x) = 1$ *exactly, the numerator is computed as* 0, *and the answer returned is* 0. *Not one digit is correct, and no warning is issued.*
Now rewrite using $1 - \cos x = 2\sin^2(x/2)$:

$$f(x) = \frac{2\sin^2(x/2)}{x^2}.$$

At $x = 10^{-8}$ *this evaluates* $\sin(5 \times 10^{-9}) \approx 5 \times 10^{-9}$ *to full relative accuracy, squares it, and returns* $2(2.5 \times 10^{-17})/10^{-16} = 0.5$. *Same function, same arithmetic, sixteen correct digits instead of none. Nothing was subtracted.*

Key Idea

Cancellation does not create error; it *exposes* error that was already there. The subtraction $1 - \mathrm{fl}(\cos x)$ is performed exactly. What ruins the result is that the rounding error already present in $\mathrm{fl}(\cos x)$, harmless at relative size 10^{-16} against a value of 1, becomes the whole of the answer once the value of 1 is cancelled away. The cure is never to form the difference, which usually means rewriting the formula rather than improving the arithmetic.

Figure 17.2 shows the same lesson on the quadratic formula, where it costs one line to fix. Both roots of $x^2 - 10^8 x + 1$ are built from b and $\sqrt{b^2 - 4}$, which agree in their first fifteen digits; adding them is harmless and subtracting them is not.

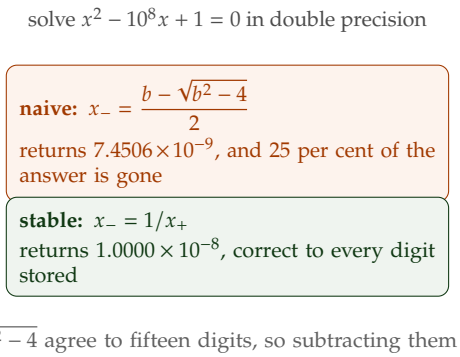


Figure 17.2. Catastrophic cancellation, and a one-line cure. Both roots of $x^2 - 10^8 x + 1$ are computed from the same two quantities, b and $\sqrt{b^2 - 4}$, which in double precision agree in their first fifteen digits. Adding them, for the large root, is harmless. Subtracting them destroys every digit they share and leaves only the noise beneath, so the small root comes back with a relative error of a quarter. Nothing is wrong with the formula, the arithmetic, or the conditioning of the problem: the roots are perfectly well determined by the coefficients. The instability is in the *algorithm*, and replacing the subtraction with $x_- x_+ = 1$ removes it entirely. That distinction, between an ill-conditioned problem and an unstable method, is what this chapter is for.

17.3 Conditioning: what belongs to the problem

Definition 17.3 (Condition number of a problem)**.** *Let f be a map and x an input. The* relative condition number *of f at x is*

$$\kappa(f,x) \;=\; \lim_{\delta\to 0}\,\sup_{\|\Delta x\|\le\delta}\; \frac{\big\|f(x+\Delta x)-f(x)\big\| \,/\, \big\|f(x)\big\|}{\|\Delta x\| \,/\, \|x\|},$$

the worst factor by which a small relative change in the input is magnified in the output. A problem with large κ is called ill-conditioned.

Theorem 17.4 (Conditioning of a linear system)**.** *Let A be invertible, $Ax=b$ and $A(x+\Delta x)=b+\Delta b$. Then*

$$\frac{\|\Delta x\|}{\|x\|} \;\le\; \kappa(A)\,\frac{\|\Delta b\|}{\|b\|}, \qquad \kappa(A)=\|A\|_2\,\big\|A^{-1}\big\|_2=\frac{\sigma_1}{\sigma_n},$$

and there are choices of b and Δb for which equality holds.

Proof. Subtracting, $A\,\Delta x=\Delta b$, so $\Delta x=A^{-1}\Delta b$ and $\|\Delta x\|\le\big\|A^{-1}\big\|\,\|\Delta b\|$. Also $b=Ax$ gives $\|b\|\le\|A\|\,\|x\|$, that is $1/\|x\|\le\|A\|\,/\|b\|$. Multiplying the two inequalities gives the bound, and $\|A\|_2\,\big\|A^{-1}\big\|_2=\sigma_1/\sigma_n$ by Proposition 16.9 applied to A and to A^{-1}, whose singular values are the reciprocals. For equality take $b=\sigma_1 u_1$, so that $x=v_1$ has the largest possible norm for its right-hand side, and Δb along u_n, which A^{-1} magnifies most. □

Key Idea

The condition number is a property of A and of the problem, not of any algorithm. If $\kappa(A)=10^{12}$ then the data itself, known only to sixteen digits, determines the answer to at most four; no method whatever can recover more, and a method that appears to is reporting noise. This is the sense in which Chapter 10 could call a matrix badly conditioned without mentioning how anything was computed.

17.4 Stability: what belongs to the algorithm

Definition 17.5 (Backward error and backward stability)**.** *Let $\hat{y}$ be what an algorithm returns for the problem f at input x. A* backward error *for $\hat{y}$ is any Δx with $f(x+\Delta x)=\hat{y}$; the backward error is the smallest $\|\Delta x\|/\|x\|$ among these. The algorithm is* backward stable *if for every input it returns the exact answer to a problem whose data differs relatively by $O(\varepsilon)$:*

$$\hat{y} = f(\tilde{x}) \quad \text{for some } \tilde{x} \text{ with } \frac{\|\tilde{x}-x\|}{\|x\|} = O(\varepsilon).$$

Figure 17.3 is that definition in one picture. Instead of measuring how far the computed output sits from the true one, which is the forward error, it asks how far the input would have to move for the computed output to be exactly right.

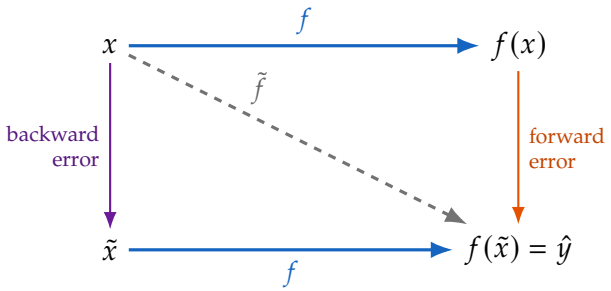


Figure 17.3. Backward error analysis in one picture. The dashed arrow is what the machine actually computed. Rather than measure how far its output is from the true output, which is the forward error on the right, we ask how far the input would have to move for that output to be exactly right, which is the backward error on the left. An algorithm is backward stable when the left-hand arrow is always short, and Theorem 17.6 converts that into a statement about the right-hand one.

Theorem 17.6 (Accuracy is stability times conditioning)**.** *If an algorithm for f is backward stable, then its forward relative error satisfies*

$$\frac{\|\hat{y}-f(x)\|}{\|f(x)\|} = O\big(\kappa(f,x)\,\varepsilon\big).$$

Proof. Backward stability gives $\hat{y} = f(\tilde{x})$ with $\|\tilde{x} - x\| / \|x\| = O(\varepsilon)$. Applying Definition 17.3 to the perturbation $\Delta x = \tilde{x} - x$,

$$\frac{\|f(\tilde{x}) - f(x)\|}{\|f(x)\|} \;\le\; \big(\kappa(f,x) + o(1)\big)\frac{\|\tilde{x} - x\|}{\|x\|} \;=\; O\big(\kappa(f,x)\varepsilon\big). \qquad \square$$

Key Idea

That identity is the whole of the subject in one line: *accuracy is stability times conditioning*. The two factors are independent and both are necessary. A backward stable algorithm on an ill-conditioned problem returns a poor answer and there is nothing to be done about it. A backward unstable algorithm on a well-conditioned problem also returns a poor answer, and that is entirely the algorithm's fault. Only the second kind of failure is worth being annoyed about, and distinguishing them is what the framework is for.

17.5 The book's warnings, reread

Every computational caution in this book is now a statement about one of the two factors.

Elimination, and the growth factor

The computed factors from Gaussian elimination with partial pivoting satisfy $\hat{L}\hat{U} = A + \Delta A$ with

$$\frac{\|\Delta A\|}{\|A\|} = O(\rho\, n\, \varepsilon), \qquad \rho = \frac{\max_{i,j,k} \left|a_{ij}^{(k)}\right|}{\max_{i,j} \left|a_{ij}\right|},$$

where ρ is the *growth factor*, the largest entry appearing anywhere during the elimination relative to the largest entry at the start. So elimination is backward stable exactly when ρ stays modest.

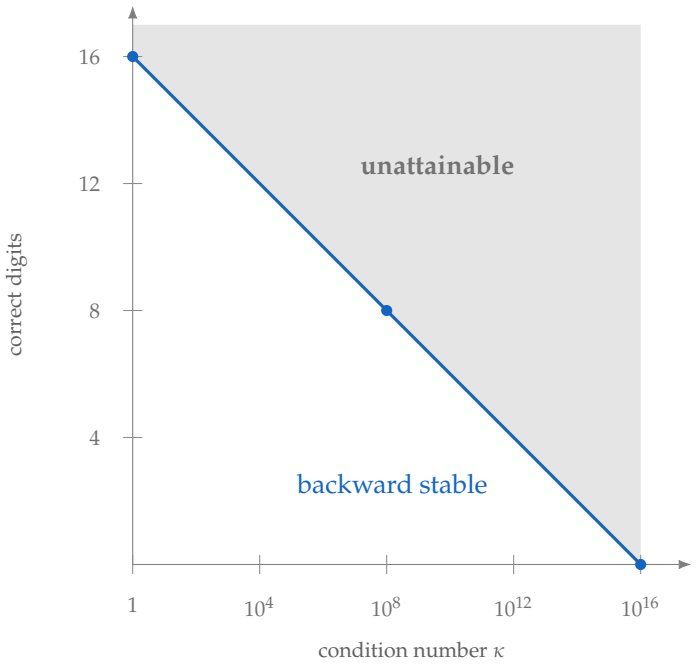


Figure 17.4. Theorem 17.6 plotted. Every digit of condition number costs a digit of accuracy, whatever the method: the shaded region above the line cannot be entered. A backward stable algorithm sits on the line, and by the time κ reaches 10^{16} the double precision input determines nothing at all. Note what the picture does *not* say: it does not promise that an algorithm sits on the line, only that none sits above it.

Caution

Partial pivoting does not bound ρ by anything useful. Wilkinson's example is the matrix with 1 on the diagonal, -1 below it, and 1 throughout the last column:

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & 1 \end{pmatrix}.$$

Partial pivoting makes no swaps, since every candidate pivot has modulus 1, and each elimination step doubles the last column: for size n the growth factor is exactly 2^{n-1}. At $n = 60$ that is larger than $1/\varepsilon$ and the computed factorisation is worthless.

And yet in practice ρ is almost always small, growing like a low power of n on matrices that arise from applications or at random, so elimination is used

everywhere and works. Why partial pivoting behaves so much better than its worst case permits is a genuinely open question, and one of the more famous embarrassments of numerical analysis.

Least squares: the two routes compared

Chapter 13 gave two algorithms and asserted that one was wrong. Now the statement is quantitative. Householder QR is backward stable without qualification, so by Theorem 17.6 solving $R\hat{x} = Q^t b$ produces a relative error of order $\kappa(A)\varepsilon$. Forming the normal equations replaces the problem by one whose condition number is $\kappa(A)^2$, by Proposition 16.11, so even a perfectly stable solve on A^tA yields an error of order $\kappa(A)^2\varepsilon$.

Example 17.7 (What the difference is worth)**.** *Suppose $\kappa(A) = 10^8$, which is unremarkable for a design matrix with badly scaled columns. In double precision, with $\varepsilon \approx 10^{-16}$:*

$$\text{QR: } \kappa\varepsilon \approx 10^{-8} \text{ (about eight correct digits)}, \qquad \text{normal equations: } \kappa^2\varepsilon \approx 1 \text{ (none)}.$$

The Läuchli matrix in the Caution of Section 13.4 is the extreme case, where A^tA becomes exactly singular in floating point while A has independent columns throughout.

Three instances of one principle

Key Idea

Do not pass through a worse-conditioned intermediate. That single rule generates every remaining warning in the book.

- Do not form A^tA: it squares the condition number (Chapter 13). Factor instead.
- Do not form A^{-1} to solve $Ax = b$: it discards the elimination and costs three times as much (Chapter 4). Keep the factorisation.
- Do not form the characteristic polynomial to find eigenvalues: the roots of a polynomial are a wildly ill-conditioned function of its coefficients, as Wilkinson's degree-twenty example shows (Chapter 9). Work on the matrix.
- Do not diagonalise as PDP^{-1} when P may be ill-conditioned (Chapter 10). Use the unitary factorisations of Chapter 14, which cannot amplify anything because Proposition 12.15 says they preserve every length exactly.

17.6 What things cost

The factorisations of this book all cost a constant times n^3, and the constants matter in practice. For an $n \times n$ system, or an $m \times n$ matrix with $m \geq n$, the leading terms are approximately

LU with partial pivoting	$\frac{2}{3}n^3$
Cholesky, for symmetric positive definite A	$\frac{1}{3}n^3$
Householder QR	$2mn^2 - \frac{2}{3}n^3$
symmetric eigenvalues, or the SVD	a larger multiple of n^3

counting multiplications and additions together. Cholesky, which is Proposition 15.6, is half the cost of LU because symmetry halves the work, which is the general lesson: structure is the only thing that ever makes a computation cheaper.

Insight

The cubic term is the wall. At $n = 10^4$ a factorisation is a few minutes of arithmetic; at $n = 10^6$ it is 10^{18} operations and 10^{12} words of storage, and neither is available. Yet systems of that size are routine in scientific computing and in machine learning, and they are solved. The escape is never a better constant. It is to stop forming the matrix at all: to exploit sparsity, symmetry, low rank, or the availability of a fast product $x \mapsto Ax$, and then to build the answer by repeated multiplication rather than by factorisation. Those are the iterative and Krylov methods, and Chapter 16 has already supplied the tool that decides when a low-rank surrogate will do. It is where this subject continues.

Open Problem 17.8 (Why does partial pivoting never fail?)**.** *Gaussian elimination with partial pivoting is the algorithm that solves linear systems in every piece of numerical software ever shipped. Its backward error is governed by the* growth factor*, the ratio of the largest entry appearing anywhere during elimination to the largest entry of the original matrix.* Why is the growth factor never large in practice, when the worst case is 2^{n-1}?

What is known. *Wilkinson proved the bound $\rho_n \leq 2^{n-1}$ and exhibited a matrix attaining it: put ones on the diagonal, minus ones below it, and ones in the last column. That matrix doubles the last column at every step. Yet in decades of computation on real problems, growth factors above n are essentially never observed, and Trefethen and Schreiber's experiments suggest an average behaviour like $n^{2/3}$ for random matrices [608].* No theorem explains this. *Even for Gaussian random matrices, a proof that the growth*

factor is polynomially bounded with high probability was obtained only recently and with a large exponent, and the sharp behaviour is open.

Where the difficulty sits. *Wilkinson's example is not fragile in the way one hopes: it is a well-conditioned matrix, so it cannot be dismissed as a numerical curiosity, and the exponential growth is not an artefact of a nearly singular problem. It is fragile in a different and more instructive way. Every pivot in it is an exact tie, $|a_{ik}| = |a_{kk}| = 1$, and partial pivoting resolves the tie in favour of the row it already has; break any one of those ties the other way and the growth collapses at once. What appears to save practice is that the growth requires a very particular sign pattern to persist through every step of elimination, and a random matrix loses that pattern almost immediately. Turning "loses that pattern" into a theorem requires understanding the joint distribution of the Schur complements, which are highly dependent objects.*

Where a reader might start. *Build Wilkinson's matrix at $n = 20$ and watch the last column double, then subtract 10^{-3} from one subdiagonal entry, so that partial pivoting is forced to swap a row, and watch the growth collapse from 2^{19} to 2. Then generate random Gaussian matrices at several sizes and plot the observed growth factor against n.* The gap between the two experiments is the open problem, *and this is one of the rare cases where an afternoon of computing puts a reader exactly at the research frontier.*

Exercises

Exercise 17.9. *In IEEE double precision, what is the largest number x with $\mathrm{fl}(1 + x) = 1$? What is the gap between 2^{40} and the next float above it? Explain why the answer to the second question is not ε.*

Exercise 17.10. *The roots of $x^2 - 200x + 1 = 0$ are 199.994999875 and 0.005000125, to the digits shown; note that they sum to 200 and multiply to 1. Compute the smaller root by the usual formula $\left(200 - \sqrt{200^2 - 4}\right)/2$, carrying six significant digits throughout, and then by the algebraically equivalent $2\big/\left(200 + \sqrt{200^2 - 4}\right)$. Explain the difference.*

Exercise 17.11. *For $A = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1.0001 \end{smallmatrix}\right)$, compute $\kappa(A)$ from the singular values. Then solve $Ax = (2, 2.0001)$ and $Ax = (2, 2.0002)$ exactly and compare the relative change in x with the relative change in b. Is Theorem 17.4 tight here?*

Exercise 17.12. *Verify by hand that the* 3×3 *Wilkinson matrix* $\begin{pmatrix} 1 & 0 & 1 \\ -1 & 1 & 1 \\ -1 & -1 & 1 \end{pmatrix}$ *has growth factor* 4 *under partial pivoting, and explain why no row swap occurs.*

Exercise 17.13. *Give three double precision numbers* a, b, c *for which* $(a+b)+c \neq a+(b+c)$ *in floating point, and say which of the two answers is closer to the true sum. What does this imply about summing a long list, and what does it suggest about the order to sum it in?*

Exercise 17.14. *Consider computing* $f(x) = \sqrt{x+1} - \sqrt{x}$ *for large* x*. Show that* $\kappa(f, x)$ *stays bounded, so the problem is well conditioned, and that the obvious algorithm is nevertheless inaccurate for* $x = 10^{16}$*. Find a backward stable alternative.* Hint: multiply by a conjugate.

Exercise 17.15. *Show that* $\kappa(Q) = 1$ *for every orthogonal* Q*, and that* $\kappa(AB) \leq \kappa(A)\kappa(B)$ *in general. Use these to explain why a factorisation into orthogonal factors cannot make a problem worse conditioned, and why* PDP^{-1} *can.*

Exercise 17.16. *A problem has* $\kappa = 10^5$ *and is solved by a backward stable algorithm in double precision. Roughly how many correct digits should you expect? What if the computation is done in single precision, where* $\varepsilon \approx 6 \times 10^{-8}$*? Locate both answers on Figure 17.4.*

Exercise 17.17. *A dense* $n \times n$ *system is solved by LU. Estimate the time at* $n = 10^3, 10^4$ *and* 10^5 *on a machine sustaining* 10^{11} *operations per second, and the storage each needs in double precision. At which size does storage become the binding constraint rather than time?*

References and further reading

This chapter is a compressed account of a large subject, and the place to go next is Trefethen and Bau [607], whose lectures on conditioning, stability and backward error analysis are the source of the framing used here, including the identity that accuracy is stability times conditioning and the discussion of why partial pivoting works better than it should. Higham [320] is the comprehensive reference and carries the sharp

forms of every bound stated loosely above. Wilkinson's own account [647] remains readable, and is worth seeing for the origin of backward error analysis in the hands of someone who had computed by hand at scale. Goldberg's survey [262] of what every computer scientist should know about floating point arithmetic is the standard introduction to Section 17.1, and Kahan's writings on the design of IEEE 754 explain why the fundamental axiom can be assumed at all, which is a matter of engineering rather than mathematics and was not inevitable.